

Applied Linear Algebra Concepts in Computer and Data Science

Accompanying Software: **Visu(AL)-I**

Aleron Tham & Ng Yao Teck, National University of Singapore | Last updated 26 June 2026



Foreword

- At the time of publication, we are undergraduate students at the National University of Singapore (NUS).
- Consequently, this slide deck is primarily directed towards fellow undergraduate students at NUS who struggle to understand the geometric intuition behind linear algebra concepts. Students are expected to be currently reading or have read one of the following courses at NUS:
 - MA1508E Linear Algebra for Engineering
 - MA1513 Linear Algebra with Differential Equations
 - MA1522 Linear Algebra for Computing
 - MA2001 Linear Algebra I
 - or any other equivalent introductory linear algebra course at other autonomous universities in Singapore.
- Aleron Tham is a Computer Science major and Ng Yao Teck is a Data Science & Analytics major. We both enjoy applied mathematics and desire to leverage technology for efficient mathematical computations and visualisation of mathematical concepts.
- Consequently, we made Visu(AL)-1 to linear algebra concepts visually, in hopes of helping current and future students to understand linear algebra better. This slide deck can be used as a reference while using Visu(AL)-1.

Table of Contents

- Preliminaries
- Recap of Linear Algebra Concepts and their Geometrical Meanings
- 3D Graphics
- 2D Image Processing
- Numerical Stability (TBD)

Preliminaries

- Systems of Linear Equations, Row-Echelon Forms, Elementary Row Operations, Gauss-Jordan Elimination
- Matrix Algebra and Properties, Inverse Matrices, Properties of Determinant
- Euclidean Vector Spaces, Inner Product, L^2 norm, Distance and Angle between Vectors
- The above topics should've been covered in the early weeks of MA2001/MA1522/MA1508E. We will not go over them here, but understanding these properties well is important for understanding these notes and the applications of Linear Algebra (LA).
- The topics that should've been covered in LA classes that we will go over again briefly here include linear spans and basis, dimension, rank and kernel, orthogonality, eigenanalysis and linear transformations. The goal here is to provide geometric intuition rather than mathematical rigour.
- Note that decompositions such as LU, QR and SVD find uses in other applied LA fields, and SVD in particular does find a use in 3D geometry (finding the best fit plane for a set of points in 3D). However, these would otherwise be considered secondary for basic concepts in the geometry of Euclidean vector spaces.

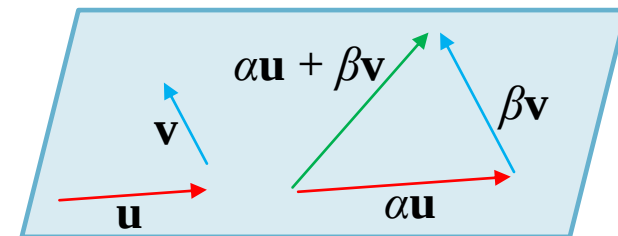
Recap: Linear combinations and spans

- Let $S = \{\mathbf{u}_1, \mathbf{u}_2, \dots, \mathbf{u}_k\}$ be a set of vectors in $\mathbb{R}^n$. A linear combination of these vectors is $c_1\mathbf{u}_1 + c_2\mathbf{u}_2 + \dots + c_k\mathbf{u}_k$, for some constants $c_1, c_2, \dots, c_k \in \mathbb{R}$.
- The span of S is the subset of $\mathbb{R}^n$ containing all the linear combinations of the vectors in S . That is, given $\mathbf{v} \in \mathbb{R}^n$,

$$\mathbf{v} \in \text{span}(S) \Leftrightarrow \exists c_1, c_2, \dots, c_k \in \mathbb{R} \{ \mathbf{v} = c_1\mathbf{u}_1 + c_2\mathbf{u}_2 + \dots + c_k\mathbf{u}_k \}.$$

- For example, let $S = \left\{ \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix} \right\}$. We have $\begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix} \in \text{span}(S)$, but $\begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} \notin \text{span}(S)$.
- Form the matrix $\mathbf{A} = (\mathbf{u}_1 \ \mathbf{u}_2 \ \dots \ \mathbf{u}_k)$. Notice that if the system $\mathbf{A}\mathbf{x} = \mathbf{v}$ is consistent (where $\mathbf{x} \in \mathbb{R}^k$), then $\mathbf{v} \in \text{span}(S)$. Intuitively, we are looking for the constants $c_1, c_2, \dots, c_k$ to find a linear combination of vectors in S that gives $\mathbf{v}$.
- The converse is also true: If $\mathbf{v} \in \text{span}(S)$, then $\mathbf{A}\mathbf{x} = \mathbf{v}$ must be consistent.
- Spans are closed under linear combination. That is,

$$\mathbf{u}, \mathbf{v} \in \text{span}(S) \wedge \alpha, \beta \in \mathbb{R} \Rightarrow \alpha\mathbf{u} + \beta\mathbf{v} \in \text{span}(S).$$



- Geometrically, the region shown is $\text{span}\{\mathbf{u}, \mathbf{v}\}$. Any linear combination of $\mathbf{u}$ and $\mathbf{v}$ lies in the span.

Questions

Suppose $\mathbf{v}$ is a linear combination of the vectors in $S = \{\mathbf{u}_1, \mathbf{u}_2, \dots, \mathbf{u}_k\}$.

- Is the linear combination for $\mathbf{v}$ necessarily unique?
- Show that if $\mathbf{A} = (\mathbf{u}_1 \ \mathbf{u}_2 \ \dots \ \mathbf{u}_k)$ and $\text{RREF}(\mathbf{A})$ has no zero rows, then $\text{span}(S) = \mathbb{R}^n$.

Answer:

- 1) If $\mathbf{0} \notin S$ but $c_1\mathbf{u}_1 + c_2\mathbf{u}_2 + \dots + c_k\mathbf{u}_k = \mathbf{0}$ and at least one of $c_1, c_2, \dots, c_k$ is non-zero, then linear combinations are not necessarily unique.
- 2) This is equivalent to the system $\mathbf{Ax} = \mathbf{v}$ being consistent for all $\mathbf{v} \in \mathbb{R}^n$.

Recap: Linear spaces (subspaces)

- A linear space (subspace) over n -dimensional real vectors is a space in which vectors are closed in the space under addition and scalar multiplication. That is, if S is a linear space and $\mathbf{a}_1, \mathbf{a}_2 \in S$, then we must have $\mathbf{a}_1 + \mathbf{a}_2 \in S$ and $c\mathbf{a}_1 \in S$, where $c \in \mathbb{R}$.
- An important corollary is that any linear space must contain the zero vector.
- Linear spaces can also be defined over $n \times m$ matrices, but we will rarely play around with these cases.
- An equivalent definition is as follows: A subset $V \subseteq \mathbb{R}^n$ is a subspace if and only if it is a linear span, $V = \text{span}(S)$, for some finite set $S \{\mathbf{u}_1, \mathbf{u}_2, \dots, \mathbf{u}_k\}$.
- To show that a subset V is a linear space, we can either find a spanning set, or show that V contains the zero vector and is closed under linear combination.
- Question: When is V not a linear space?

Geometrical interpretation of linear spaces

- The space $\{(x, y, z) : x + y + z = 0\}$ is a linear space, but the space $\{(x, y, z) : x(y + z) = 0\}$ is not, since the space is not closed under addition; $(1, 0, 0)$ and $(0, 1, 0)$ are both in the space, but their sum $(1, 1, 0)$ is not.
- Geometrically, the linear space $\{(x, y, z) : x + y + z = 0\}$ is the plane with a normal vector parallel to $(1, 1, 1)$ and passing through the origin.
- Similarly, the linear space $\{(x, y, z) : x = y = z\}$ is geometrically represented by the line with a direction vector parallel to $(1, 1, 1)$ and passing through the origin.
- The definition of the latter space is actually just a shorthand for the system $\{1x - 1y + 0z = 0, 1x + 0y - 1z = 0\}$. That is, it is the intersection of two non-parallel planes, which as one can recall from A-level maths, is a line.
- The space $\{\mathbf{0}\}$ is also a linear space. It represents the origin!
- $\mathbb{R}^3$ is also a linear space. It represents the entire space of our 3D world!

Affine spaces

- An affine space can be thought of as a ‘translated’ linear space.
- The subset $S = \{(x, y, z) : x + y + z = 1\}$ is not a linear space since it does not contain $\mathbf{0}$.
- However, let $S' = \{(x, y, z) : x + y + z = 0\}$. Check that S' is a linear space. Notice also that $(0, 0, 1) \in S$, and that for all $\mathbf{v} \in S$, $\mathbf{v} - (0, 0, 1) \in S'$.
- More generally, a subset S is an affine space if there exists a linear space S' and a constant vector $\mathbf{v}_0 \in S$ such that for all $\mathbf{v} \in S$, $\mathbf{v} - \mathbf{v}_0 \in S'$. If $\mathbf{v}_0 = \mathbf{0}$, then S is itself a linear space and we have $S \subseteq S'$.
- Geometrically, the plane $x + y + z = 1$ is a translation of the plane $x + y + z = 0$.
- Affine spaces generalise linear spaces in a way that is useful for computer graphics and other applications. The gist of it is that while linear transformations are forced to be ‘centred’ at the origin, affine transformations can be used to transform a set of vectors with a ‘centre’ wherever we want.

Recap: Basis, Dimension, Column / Row Spaces, Null Spaces, Rank / Nullity

- A basis B for a linear space S is such that B consists of linearly independent vectors and $\text{span}(B) = S$.
- The dimension of S is the number of vectors in B . Any basis must have the same dimension d .
 - If a set has fewer than d vectors, it cannot span S .
 - If a set has more than d vectors, it must be linearly dependent.
- The column or row space of a $m \times n$ matrix $\mathbf{A}$ is the subspace of $\mathbb{R}^m$ or $\mathbb{R}^n$ spanned by the columns or rows of $\mathbf{A}$.
- The null space of a $m \times n$ matrix $\mathbf{A}$ is the subspace of $\mathbb{R}^n$ containing all vectors $\mathbf{v}$ which satisfy $\mathbf{A}\mathbf{v} = \mathbf{0}$.
- The rank of a matrix is the number of linearly independent rows or columns (or equivalently, the dimension of its column space or row space), and the nullity of a matrix is the dimension of its null space.
- The rank-nullity theorem asserts that $\text{rank}(\mathbf{A}) + \text{nullity}(\mathbf{A}) = n$.
- Geometrically, the dimension of a subspace is the number of linearly independent vectors needed to specify any point in a given space. Dimension 1 is a line, dimension 2 is a plane, etc.
- While we will not cover any immediate geometrical implications of the rank-nullity theorem, it will be needed for a proof later. Do refer to the MA2001/MA1522 lecture notes for the full details if you need a refresher.

Recap: Orthogonality

- Orthogonality is a generalisation of the concept of perpendicularity in 2D and 3D, just as linear independence is a generalisation of the concept of parallelism.
- Two vectors are said to be orthogonal to each other if and only if their inner product is zero.
- A set is orthogonal if its vectors are all pairwise orthogonal, and a set is orthonormal if it is orthogonal and the vectors all have unit norm.
- A vector $\mathbf{u}$ is orthogonal to a subspace V if $\mathbf{u} \cdot \mathbf{v} = 0$ for every $\mathbf{v} \in V$. Alternatively, if S is a spanning set for V , then we have $\mathbf{u} \perp V \Leftrightarrow \mathbf{u} \in \text{Null}(\mathbf{A}^T)$, where $\mathbf{A}$ is the matrix whose columns are the column vectors in S .
- The orthogonal complement of V , denoted $V^\perp$, is the set of all vectors that are orthogonal to V . A theorem is that if $\mathbf{A}$ is the matrix defined above, then $V^\perp = \text{Null}(\mathbf{A}^T)$.
- An orthogonal (orthonormal) basis is a basis with orthogonal (orthonormal) vectors.
- A square matrix $\mathbf{M}$ is orthogonal if and only if its columns (or rows) form an orthonormal basis, or equivalently, $\mathbf{M}^T = \mathbf{M}^{-1}$.

Recap: Orthogonal Projection and the Gram-Schmidt Process

- The orthogonal projection theorem states that given a subspace V of $\mathbb{R}^n$, every vector in $\mathbb{R}^n$ can be decomposed *uniquely* as a sum of two vectors, one in V and one in $V^\perp$.
- Specifically, if $\{\mathbf{u}_1, \mathbf{u}_2, \dots, \mathbf{u}_k\}$ is an orthogonal basis for V , and $\mathbf{w} = \mathbf{w}_p + \mathbf{w}_n$ where $\mathbf{w}_p \in V$, $\mathbf{w}_n \in V^\perp$, then

$$\mathbf{w}_p = \frac{\mathbf{w} \cdot \mathbf{u}_1}{\mathbf{u}_1 \cdot \mathbf{u}_1} \mathbf{u}_1 + \frac{\mathbf{w} \cdot \mathbf{u}_2}{\mathbf{u}_2 \cdot \mathbf{u}_2} \mathbf{u}_2 + \dots + \frac{\mathbf{w} \cdot \mathbf{u}_k}{\mathbf{u}_k \cdot \mathbf{u}_k} \mathbf{u}_k.$$

- The Gram-Schmidt process converts a linearly independent set to an orthogonal set. It proceeds as follows:

$$\mathbf{v}_1 = \mathbf{u}_1$$

$$\mathbf{v}_2 = \mathbf{u}_2 - \left(\frac{\mathbf{v}_1 \cdot \mathbf{u}_2}{\|\mathbf{v}_1\|^2} \right) \mathbf{v}_1$$

$$\mathbf{v}_3 = \mathbf{u}_3 - \left(\frac{\mathbf{v}_1 \cdot \mathbf{u}_3}{\|\mathbf{v}_1\|^2} \right) \mathbf{v}_1 - \left(\frac{\mathbf{v}_2 \cdot \mathbf{u}_3}{\|\mathbf{v}_2\|^2} \right) \mathbf{v}_2$$

$$\vdots$$

$$\mathbf{v}_k = \mathbf{u}_k - \left(\frac{\mathbf{v}_1 \cdot \mathbf{u}_k}{\|\mathbf{v}_1\|^2} \right) \mathbf{v}_1 - \left(\frac{\mathbf{v}_2 \cdot \mathbf{u}_k}{\|\mathbf{v}_2\|^2} \right) \mathbf{v}_2 - \dots - \left(\frac{\mathbf{v}_{k-1} \cdot \mathbf{u}_k}{\|\mathbf{v}_{k-1}\|^2} \right) \mathbf{v}_{k-1}.$$

Recap: Eigenvalues and eigenvectors

- Given a square matrix $\mathbf{A}$ of order n and a non-zero vector $\mathbf{v} \in \mathbb{C}^n$, we say that $\mathbf{v}$ is an eigenvector of $\mathbf{A}$ if there exists $\lambda \in \mathbb{C}$ such that $\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$. In such a case, λ is known as an eigenvalue of $\mathbf{A}$.
- For example, consider the matrix $\mathbf{A} = \begin{pmatrix} 0 & 1 \\ -2 & 3 \end{pmatrix}$. Observe that $\begin{pmatrix} 0 & 1 \\ -2 & 3 \end{pmatrix} \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$ and $\begin{pmatrix} 0 & 1 \\ -2 & 3 \end{pmatrix} \begin{pmatrix} 1 \\ 2 \end{pmatrix} = \begin{pmatrix} 2 \\ 4 \end{pmatrix} = 2\begin{pmatrix} 1 \\ 2 \end{pmatrix}$. Hence, 1 and 2 are eigenvalues of $\mathbf{A}$ with respective eigenvectors $\begin{pmatrix} 1 \\ 1 \end{pmatrix}$ and $\begin{pmatrix} 1 \\ 2 \end{pmatrix}$.
- Eigenvectors are not unique as any non-zero scalar multiple of an eigenvector is also an eigenvector.
- Furthermore, it is possible that multiple linearly independent vectors are eigenvectors of a square matrix which correspond to the same eigenvalue. Consider $\mathbf{I}_2 \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$, and $\mathbf{I}_2 \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$. Hence, $\begin{pmatrix} 1 \\ 0 \end{pmatrix}$ and $\begin{pmatrix} 0 \\ 1 \end{pmatrix}$ are eigenvectors of $\mathbf{I}_2$, and both have corresponding eigenvalue 1.
- Note here that although we will only discuss matrices whose entries are all in $\mathbb{R}$, it is entirely possible that their eigenvalues and eigenvectors are in $\mathbb{C}$.

Recap: Eigenvalues and eigenvectors

- Given a square matrix $\mathbf{A}$ of order n and a non-zero vector $\mathbf{v} \in \mathbb{C}^n$, we say that $\mathbf{v}$ is an eigenvector of $\mathbf{A}$ if there exists $\lambda \in \mathbb{C}$ such that $\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$. In such a case, λ is known as an eigenvalue of $\mathbf{A}$.
- An equivalent statement to the above is that $(\lambda\mathbf{I} - \mathbf{A})\mathbf{v} = \mathbf{0}$, that is, $\mathbf{v}$ is a non-trivial solution of the homogenous system of equations $(\lambda\mathbf{I} - \mathbf{A})\mathbf{x} = \mathbf{0}$.
- Hence, the set of all solutions to the above system forms a subspace of $\mathbb{C}^n$. This subspace is also known as the eigenspace of $\mathbf{A}$ associated to λ .
- Question: Can 0 be an eigenvalue of $\mathbf{A}$? What would it imply?

Geometrical Transformations in 2D and 3D

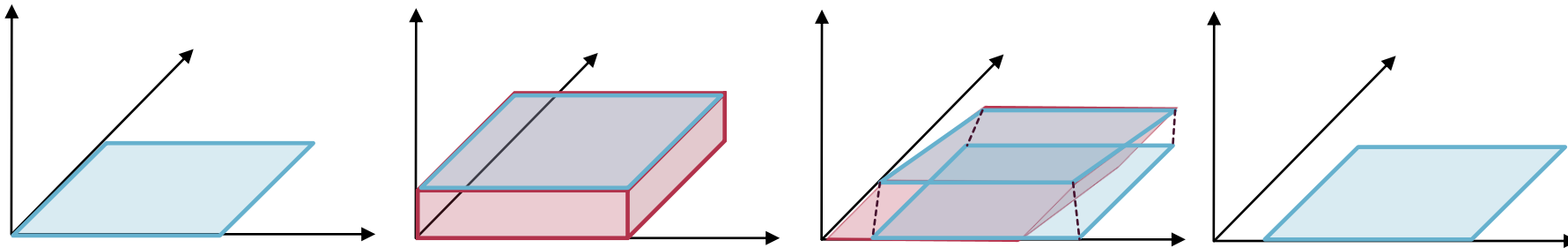


Introduction to Homogenous Coordinates: Translation of a point in 2D

- Consider a transformation which translates a point $P : (p_1, p_2)$ in the xy -plane to the point $P' : (p_1 + a_1, p_1 + a_2)$. That is, we want a translation from P to P' .
- Recall that a transformation T is linear only if $T(\mathbf{0}) = \mathbf{0}$. Hence, there is no linear transformation $\mathbb{R}^2 \rightarrow \mathbb{R}^2$ that maps the position vector of P to the position vector of P' unless $a_1 = a_2 = 0$, which is just the identity transformation.
- However, what if we move P to a 3D space? Here's the idea: We can temporarily add a third dimension so that we can map p_1 to $p_1 + a_1$ and p_2 to $p_2 + a_2$. In order to achieve this, we can add a_1 and a_2 to the third column of our transformation matrix, and make the third row of the matrix $(0 \ 0 \ 1)$, to ensure the z -coordinate remains unchanged.
- There exists a linear transformation $T : \mathbb{R}^3 \rightarrow \mathbb{R}^3$ such that $T\left(\begin{pmatrix} p_1 \\ p_2 \\ 1 \end{pmatrix}\right) = \begin{pmatrix} p_1 + a_1 \\ p_2 + a_2 \\ 1 \end{pmatrix}$. The matrix $\begin{pmatrix} 1 & 0 & a_1 \\ 0 & 1 & a_2 \\ 0 & 0 & 1 \end{pmatrix}$ is its standard matrix. Check that, for any point in $\mathbb{R}^3$, the z -coordinate remains unchanged when T is applied to it.
- If we want to compose multiple 2D affine transformations, we can chain up multiple 3×3 matrices representing linear transformations with homogenous coordinates. Notice that only the x and y -coordinates will change due to these transformations, while the z -coordinate will always remain constant.

Representing Affine Transformations as Linear Transformations using Homogenous Coordinates

- We have just seen that a square matrix of order 3 can be used to represent translations in $\mathbb{R}^2$. Geometrically, a translation in an $\mathbb{R}^2$ space can be achieved by extruding the space into $\mathbb{R}^3$ using prism extrusion, followed by a shearing in this new $\mathbb{R}^3$ space, followed by a projection back into the original $\mathbb{R}^2$ space, as shown below:



- In general, for a translation in $\mathbb{R}^n$, an $(n + 1)$ -dimensional space is needed to represent this translation as a linear transformation. We say that translation in $\mathbb{R}^n$ is an *affine transformation* in $\mathbb{R}^n$. All linear transformations are affine transformations, but not all affine transformations are linear transformations.
- More generally, if $T : \mathbb{R}^n \rightarrow \mathbb{R}^n$ is an affine transformation, it can be expressed as the composition of a translation and a linear transformation, and there exists a linear transformation $T' : \mathbb{R}^{n+1} \rightarrow \mathbb{R}^{n+1}$ such that $T'\left(\begin{pmatrix} \mathbf{p} \\ 1 \end{pmatrix}\right) = \begin{pmatrix} T(\mathbf{p}) \\ 1 \end{pmatrix}$ for any vector $\mathbf{p} \in \mathbb{R}^n$.

Why keep the fourth coordinate?

- Since objects are only being displayed in three dimensions, and the fourth coordinate of any homogenous vector in a 3D space is always 1, why do we need it after the transformation?
- Indeed, in the above case, the transformation matrix would be 3×4 instead of 4×4 , which does save space. The output vector would be in $\mathbb{R}^3$ instead of $\mathbb{R}^4$ as well.
- However, notice that to apply another affine transformation, or to compose affine transformations, they are only well-defined if we keep the matrices square.
- Furthermore, although this is out of our scope, there are also *perspective matrices* in 3D graphics pipelines that do actually use the fourth row of the transformation matrix.
- For performing vector geometry on the actual objects, indeed, we tend to disregard the fourth coordinate of each vertex making up an object. However, since the fourth dimension does have a meaning for 3D graphics pipelines, we rarely throw it out completely.

Reflection of a point about a line in 2D/3D

- Consider the line $L : \mathbf{r} = \mathbf{a} + \lambda \mathbf{d}$, where $\lambda \in \mathbb{R}$ and $\mathbf{d} \neq \mathbf{0}$, and the point P with position vector $\mathbf{p}$ in the xyz -space, and let P' denote the reflection of P in L . What is the position vector of P' ?
- This is of course a standard A-level problem. Assuming $\mathbf{f}$ is the position vector of the foot of perpendicular from P to L , we know that $\mathbf{p}' = 2\mathbf{f} - \mathbf{p}$. We also know using the projection formula that $\mathbf{f} = \frac{((\mathbf{p} - \mathbf{a}) \cdot \mathbf{d})\mathbf{d}}{\|\mathbf{d}\|^2} + \mathbf{a}$.
- For the 3D case, if we let $\mathbf{a} = \begin{pmatrix} a_1 \\ a_2 \\ a_3 \end{pmatrix}$, and denote $\mathbf{d}$ and $\mathbf{p}$ similarly, the position vector of $\mathbf{p}'$ can be expressed as
$$\begin{pmatrix} (cd_1^2 - 1)p_1 + cd_1d_2p_2 + cd_1d_3p_3 + 2a_1 - cd_1(a_1d_1 + a_2d_2 + a_3d_3) \\ cd_1d_2p_1 + (cd_2^2 - 1)p_2 + cd_2d_3p_3 + 2a_2 - cd_2(a_1d_1 + a_2d_2 + a_3d_3) \\ cd_1d_3p_1 + cd_2d_3p_2 + (cd_3^2 - 1)p_3 + 2a_3 - cd_3(a_1d_1 + a_2d_2 + a_3d_3) \end{pmatrix}, \text{ where } c = \frac{2}{d_1^2 + d_2^2 + d_3^2}.$$
- The 2D case should be easily deducible.
- Check that there is no linear transformation $\mathbb{R}^3 \rightarrow \mathbb{R}^3$ that maps $\mathbf{p}$ to $\mathbf{p}'$ unless $\mathbf{a} = k\mathbf{d}$ for some constant $k \in \mathbb{R}$.

Reflection of a point about a line in 2D/3D

- Using homogenous coordinates, we obtain
$$\begin{pmatrix} cd_1^2 - 1 & cd_1d_2 & cd_1d_3 & 2a_1 - cd_1(a_1d_1 + a_2d_2 + a_3d_3) \\ cd_1d_2 & cd_2^2 - 1 & cd_2d_3 & 2a_2 - cd_2(a_1d_1 + a_2d_2 + a_3d_3) \\ cd_1d_3 & cd_2d_3 & cd_3^2 - 1 & 2a_3 - cd_3(a_1d_1 + a_2d_2 + a_3d_3) \\ 0 & 0 & 0 & 1 \end{pmatrix}$$
 for the 3D case and
$$\begin{pmatrix} bd_1^2 - 1 & bd_1d_2 & 2a_1 - bd_1(a_1d_1 + a_2d_2) \\ bd_1d_2 & bd_2^2 - 1 & 2a_2 - bd_2(a_1d_1 + a_2d_2) \\ 0 & 0 & 1 \end{pmatrix}$$
 for the 2D case, where $b = \frac{2}{d_1^2 + d_2^2}$.
- However, in computer graphics it is common to set $\|\mathbf{d}\| = 1$ for computational simplicity. For the special case $\mathbf{a} = \mathbf{0}$ and $\|\mathbf{d}\| = 1$, we can rewrite the 2D case in standard coordinates as the 2×2 matrix
$$\begin{pmatrix} 2d_1^2 - 1 & 2d_1d_2 \\ 2d_1d_2 & 2d_2^2 - 1 \end{pmatrix}.$$
- The latter matrix is known as the 2×2 Householder matrix, and represents a Householder transformation, which is the reflection of a vector in $\mathbb{R}^n$ about some subspace of $\mathbb{R}^n$ with dimension $(n - 1)$.

Geometrical interpretation of eigenvalues and eigenvectors: Reflection about a line in 2D

- The determinant of a translation matrix is always 1, so we can ignore the effects of translation for eigenanalysis. This applies in general, so most affine transformations have the same determinant as their linear equivalents.
- Consider the reflection matrix $\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$. Its characteristic equation is $\lambda^2 - 1 = 0$, so the eigenvalues are 1 and -1 . Corresponding eigenvectors are $\begin{pmatrix} 1 \\ 1 \end{pmatrix}$ and $\begin{pmatrix} -1 \\ 1 \end{pmatrix}$. However, what is the actual geometric meaning?
- We have $\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} y \\ x \end{pmatrix}$. The transformation $T : \mathbb{R}^2 \rightarrow \mathbb{R}^2$, given by $T((x, y)) = (y, x)$, is a linear transformation (check it!) and its standard matrix is $\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$. In particular, $T(\mathbf{v})$ is a reflection of the vector $\mathbf{v}$ about the line $y = x$.
- The eigenvalue of 1 and corresponding eigenvector $\begin{pmatrix} 1 \\ 1 \end{pmatrix}$ indicates that any vector in $\text{span}\left\{\begin{pmatrix} 1 \\ 1 \end{pmatrix}\right\}$ is invariant under T (geometrically, a reflection of a line in itself should be the same line).
- The eigenvalue of -1 and corresponding eigenvector $\begin{pmatrix} -1 \\ 1 \end{pmatrix}$ indicates that any vector in $\text{span}\left\{\begin{pmatrix} -1 \\ 1 \end{pmatrix}\right\}$ maps to its negative under T (geometrically, a reflection of L_2 in L_1 , where L_2 is perpendicular to L_1 , should just be L_2 in the opposite direction).

Exercise

- Find the eigenvalues of the matrix $\mathbf{M} = \frac{1}{49} \begin{pmatrix} +41 & -12 & -24 \\ -12 & +31 & -36 \\ -24 & -36 & -23 \end{pmatrix}$ as well as corresponding eigenbases.

One possible answer: Eigenvalue 1 with basis $\left\{ \begin{pmatrix} 0 \\ 2 \\ -1 \end{pmatrix}, \begin{pmatrix} 3 \\ 0 \\ -1 \end{pmatrix} \right\}$, eigenvalue -1 with basis $\left\{ \begin{pmatrix} 2 \\ 3 \\ 6 \end{pmatrix} \right\}$

Exercise

- Hence, identify the linear transformation represented by $\mathbf{M}$, and explain the geometric interpretation of the eigenvalues and corresponding eigenbases.

Answer: $\mathbf{M}$ represents a reflection about the plane $2x + 3y + 6z = 0$. The eigenbasis corresponding to eigenvalue 1 specifies two linearly independent directions which are invariant under the reflection, and the eigenbasis corresponding to eigenvalue -1 specifies the normal to the plane. Indeed, the cross product of the two direction vectors should produce a vector parallel to the normal vector.

Reflection of a point about a plane in 3D

- Again, this problem can be solved by any good A-level student. Here we will just state the result: To reflect the point $P : (p_1, p_2, p_3)$ in the plane $\Pi : ax + by + cz = d$, the transformation matrix using homogenous coordinates is:

$$\frac{1}{a^2 + b^2 + c^2} \begin{pmatrix} b^2 + c^2 - a^2 & -2ab & -2ac & 2ad \\ -2ab & a^2 + c^2 - b^2 & -2bc & 2bd \\ -2ac & -2bc & a^2 + b^2 - c^2 & 2cd \\ 0 & 0 & 0 & a^2 + b^2 + c^2 \end{pmatrix}$$

- Or if we assume that $\left\| \begin{pmatrix} a \\ b \\ c \end{pmatrix} \right\| = 1$:
$$\begin{pmatrix} 1 - 2a^2 & -2ab & -2ac & 2ad \\ -2ab & 1 - 2b^2 & -2bc & 2bd \\ -2ac & -2bc & 1 - 2c^2 & 2cd \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

- Again, in the special case $d = 0$, the latter matrix can be written in standard coordinates as a 3×3 matrix by omitting the last row and last column. This 3×3 matrix would also be a Householder matrix.

Exercise

Without using the general formula shown earlier, find the equation of the plane P such that a reflection in P is given

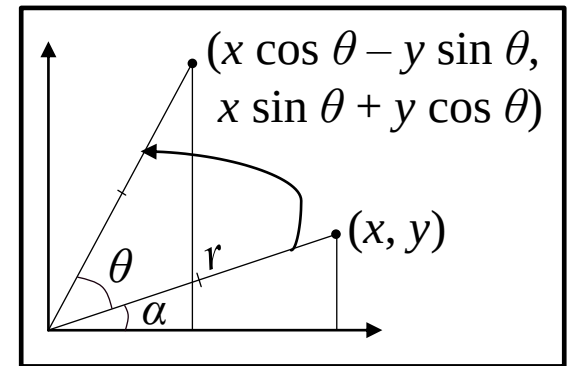
by the matrix $\mathbf{M} = \begin{pmatrix} +0.64 & -0.48 & -0.60 & +0.72 \\ -0.48 & +0.36 & -0.80 & +0.96 \\ -0.60 & -0.80 & +0.00 & +1.20 \\ +0.00 & +0.00 & +0.00 & +1.00 \end{pmatrix}$.

Trace and Determinant of Square Transformation Matrices

- Recall that for any square matrix $\mathbf{A}$, the sum of its eigenvalues is $\text{tr}(\mathbf{A})$, and the product of eigenvalues is $\det(\mathbf{A})$.
- These results follow readily from the fact that any square matrix is similar to its Jordan form (for diagonalisable matrices, the Jordan form is simply the usual diagonal matrix of eigenvalues). External reading is advised for those interested in the details.
- This has some implications for certain transformation matrices.
- Question: What are the determinants of the following matrices representing affine transformations?
 - (a) The 3×3 matrix representing a reflection about a line in 2D
 - (b) The 4×4 matrix representing a reflection about a line in 3D
 - (c) The 4×4 matrix representing a projection onto a plane in 3D

Introduction to Rotations

- Consider rotating the point P in the xy -plane by an angle θ about the origin.
- This is a linear transformation that maps (x, y) to $(x \cos \theta - y \sin \theta, x \sin \theta + y \cos \theta)$.
- To see this, set $x = r \cos \alpha$ and $y = r \sin \alpha$, where $r^2 = x^2 + y^2$ and $\tan \alpha = \frac{y}{x}$, and use the trigonometric addition formulae to expand and simplify $[r \cos(\alpha + \theta), r \sin(\alpha + \theta)]$.
- The standard matrix is $\mathbf{R}(\theta) = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix}$.
- The eigenvalues of $\mathbf{R}(\theta)$ satisfy the equation $\lambda^2 - (2 \cos \theta)\lambda + 1 = 0$, that is, $\lambda = \cos \theta \pm i \sin \theta$.
- Except when θ is a multiple of 180° (including 0°), the eigenvalues of $\mathbf{R}(\theta)$ are complex numbers! However, the trace of $\mathbf{R}(\theta)$ is always $2 \cos \theta$, which we can use to deduce the angle together with whether the sine of this angle is positive or negative.
- Rotations in 3D are a little more complicated, as we will see in the following pages.
- As with other linear transformations, using homogenous coordinates, we can represent an affine rotation as a composition of a translation and a linear rotation. This is true for both 2D and 3D rotations, but we will consider only the linear cases for simplicity.



Basic 3D Rotations

- We have just seen 2D rotations. We can also consider rotations in 3D.
- The xy -plane has been extruded into a 3D space by including a perpendicular z -axis. Hence, any point now has a z -coordinate, and so we can think of any rotation within the plane parallel to the xy -plane, that is, perpendicular to the z -axis. Here, we call the z -axis the *angle of rotation*.
- Similar logic applies to the x -axis and the y -axis. Since one coordinate always remains invariant under these three basic rotations, the standard rotation matrices are:

$$\mathbf{R}_x(\theta) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta \\ 0 & \sin \theta & \cos \theta \end{pmatrix}, \mathbf{R}_y(\theta) = \begin{pmatrix} \cos \theta & 0 & \sin \theta \\ 0 & 1 & 0 \\ -\sin \theta & 0 & \cos \theta \end{pmatrix}, \mathbf{R}_z(\theta) = \begin{pmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{pmatrix}.$$

- Note that the sine terms are in alternating signs because of the complications of coupling a right-handed coordinate system with a Cartesian (x, y, z) coordinate convention.
- Recall from A-level math (or physics) that $\mathbf{i} \times \mathbf{j} = \mathbf{k}$, $\mathbf{j} \times \mathbf{k} = \mathbf{i}$ and $\mathbf{k} \times \mathbf{i} = \mathbf{j}$. While the xy -plane and yz -plane have their coordinates specified in the correct order, the xz -plane has the opposite order (since $\mathbf{i} \times \mathbf{k} = -\mathbf{j}$), and so the rotation is by the angle $-\theta$ instead.

General 3D Rotations

- Intuitively, we can compose basic rotations to form general rotations. Indeed, a common way to do so is to rotate about the coordinate axes in the order of the x-axis, y-axis and z-axis. The matrix multiplication is as follows:

$$\mathbf{R} = \mathbf{R}_z(\alpha) \mathbf{R}_y(\beta) \mathbf{R}_x(\gamma) = \begin{pmatrix} \cos \alpha \cos \beta & \cos \alpha \sin \beta \sin \gamma - \sin \alpha \cos \gamma & \cos \alpha \sin \beta \cos \gamma + \sin \alpha \sin \gamma \\ \sin \alpha \cos \beta & \sin \alpha \sin \beta \sin \gamma + \cos \alpha \cos \gamma & \sin \alpha \sin \beta \cos \gamma - \cos \alpha \sin \gamma \\ -\sin \beta & \cos \beta \sin \gamma & \cos \beta \cos \gamma \end{pmatrix}.$$

- Conversely, given any rotation matrix $\mathbf{R}$, we can solve for the angles α , β and γ according to the above convention.
- However, it's usually more useful to determine the axis of rotation of a general rotation directly. For now, we shall assume that the axis of rotation does actually exist (barring a specific edge case). Any point that lies on this axis should be invariant under the rotation, and so we should have an eigenvalue of 1 for every rotation matrix $\mathbf{R}$.
- In fact, we will formally prove this on the next page.

General 3D Rotations

- Notice that $\mathbf{R}_x(\theta)$, $\mathbf{R}_y(\theta)$ and $\mathbf{R}_z(\theta)$ are orthogonal matrices. Intuitively, their respective inverses are obtained by replacing θ by $-\theta$, and one can verify that these inverses are the same as the transposes of the original matrices. One can also verify that the determinants of these matrices are all 1.
- Since the product of orthogonal matrices of the same size is also orthogonal, and if $\mathbf{AB}$, $\det(\mathbf{A})$ and $\det(\mathbf{B})$ are well-defined, then $\det(\mathbf{AB}) = \det(\mathbf{A})\det(\mathbf{B})$, then $\mathbf{R}$ is also orthogonal with $\det(\mathbf{R}) = 1$.
- We now use some identities about matrix transposes and determinants, and the fact that $\mathbf{R}$ is of order 3.
- Hence $\det(\mathbf{R} - \mathbf{I}) = \det(\mathbf{R}^T)\det(\mathbf{R} - \mathbf{I}) = \det(\mathbf{I} - \mathbf{R}^T) = \det((\mathbf{I} - \mathbf{R})^T) = \det(\mathbf{I} - \mathbf{R}) = (-1)^3\det(\mathbf{R} - \mathbf{I}) = -\det(\mathbf{R} - \mathbf{I})$.
- Hence we conclude that 1 is an eigenvalue of every 3×3 general rotation matrix $\mathbf{R}$.
- As with 2D rotations, the other two eigenvalues are complex conjugates and their product must be 1.
- Question: Based on the above facts, when do we fail to define a specific axis of rotation?

Determining the axis and angle of general 3D rotations

- To determine the axis of rotation for a 3×3 general rotation matrix $\mathbf{R}$, suppose $\mathbf{R}$ is **not** symmetric. Consider also a vector $\mathbf{x}$ parallel to the axis of rotation, so that $\mathbf{R}\mathbf{x} = \mathbf{R}^T\mathbf{x}$, i.e. $(\mathbf{R} - \mathbf{R}^T)\mathbf{x} = \mathbf{0}$. This system can be solved by normal means.
- Note that $\mathbf{R}$ is symmetric if and only if the angle of rotation is either 0° or 180° (modulo 360°), since these are the only angles that give matrices that are their own inverses.
- In the former case, $\mathbf{R} = \mathbf{I}$ and the axis of rotation is undefined since the dimension of the eigenspace is 3 and not 1.
- In the latter case, this is equivalent to reflection about a line! Either use the formula shown earlier, or observe that $\mathbf{R}(\mathbf{I} + \mathbf{R}) = \mathbf{R} + \mathbf{R}^2 = \mathbf{R} + \mathbf{R}\mathbf{R}^T = \mathbf{I} + \mathbf{R}$, and since the eigenvalues must be 1, -1 and -1 , by rank-nullity theorem, $\text{rank}(\mathbf{I} + \mathbf{R}) = 3 - 2 = 1$. That is, any non-zero column of $\mathbf{I} + \mathbf{R}$ is parallel to the axis of rotation.

Determining the axis and angle of general 3D rotations

- The angle of rotation of $\mathbf{R}$, θ , satisfies $\cos \theta = \frac{\text{tr}(\mathbf{R}) - 1}{2}$ and $\sin \theta = -\frac{\text{tr}(\mathbf{NR})}{2}$, where if $\mathbf{n} = \begin{pmatrix} n_1 \\ n_2 \\ n_3 \end{pmatrix}$ is a unit vector parallel to the axis of rotation, then $\mathbf{N} = \begin{pmatrix} 0 & -n_3 & n_2 \\ n_3 & 0 & -n_1 \\ -n_2 & n_1 & 0 \end{pmatrix}$.
- Details are available in this paper*. (Due to lack of time before the milestone, I have yet to abridge the explanation given in the paper; it essentially involves cross products and the orthogonal projection theorem.)

* <https://arxiv.org/pdf/1810.02999>

Challenge! A result on 3D rotations

Suppose the angle of rotation of a 3D rotation matrix $\mathbf{R}$ is not a multiple of 360° (including 0°). Prove that the axis of rotation exists and can be uniquely determined from the entries of $\mathbf{R}$.

Note that we have previously *assumed* only its existence; we now ask for a *formal proof* of existence **and** uniqueness.

Sketch of Proof

- We first prove the case where the rotation axis passes through the origin, and so $\mathbf{R}$ can be written as a 3×3 matrix.
- Firstly, 1 is an eigenvalue of every 3×3 rotation matrix (see earlier proof). Since the angle of rotation is not a multiple of 360° (including 0°), $\mathbf{R} \neq \mathbf{I}$, so the dimension of the column space (or row space) of $\mathbf{R} - \mathbf{I}$ is at least 1. That is, the geometric multiplicity of the eigenvalue 1 is at most 2.
- However, a geometric multiplicity of 2 would imply an algebraic multiplicity of at least 2. This is impossible, as:
 - The product of eigenvalues of $\mathbf{R}$ is 1 since $\mathbf{R}$ is orthogonal. An algebraic multiplicity of 2 leads to the third eigenvalue also being 1, which is a contradiction.
 - An algebraic multiplicity of 3 implies that the characteristic polynomial of $\mathbf{R}$ is $(\lambda - 1)^3$, and so $\text{tr}(\mathbf{R}) = 3$ by Vieta's formula. But since $\mathbf{R}$ is orthogonal, all rows and columns must be unit vectors. The only possibility is that $\mathbf{R} = \mathbf{I}$; contradiction.
- Hence, the geometric multiplicity of 1 must be 1, that is, the dimension of its corresponding eigenspace is 1. Geometrically, it represents a line containing all points invariant under the rotation, i.e. the axis of rotation exists. Uniqueness is trivial.
- In the 4×4 matrix case (for affine rotations), we are also guaranteed one and only one axis so long as $\mathbf{R}$ is not a translation matrix. This time, we solve the system $\mathbf{R}\mathbf{x} + \mathbf{t} = \mathbf{x}$, i.e. $(\mathbf{I} - \mathbf{R})\mathbf{x} = \mathbf{t}$. The fourth row of $\mathbf{R}$ is $(0 \ 0 \ 0 \ 1)$, and the fourth entries of $\mathbf{x}$ and $\mathbf{t}$ are both 1, so a solution for $\mathbf{x}$ exists.
- For uniqueness, suppose that two different axes existed. Then, their direction vectors must both lie in $\ker(\mathbf{I} - \mathbf{R})$. Then, the set of all points invariant under the rotation would be a subspace of dimension 2 (or a plane), which is a contradiction.

So how does Visu(AL)-I actually work?

- Firstly, every object has a set of vertices in 3D, and a ‘matrix stack’ that initially only contains the identity matrix. Its ‘current matrix’ is also the identity matrix.
- The formulas for converting translations, reflections, rotations and the like are converted to matrix form, and the ‘matrix stack’ and ‘current matrix’ are updated accordingly.
- The new transformation is applied to the object vertex-wise. Once the object’s vertex coordinates have been updated, the changes are displayed on the OpenGL panel.
- Another object is also created upon the first transformation of an object – the ‘shadow object’. It represents the last position of the current object, and is drawn as a translucent object. Subsequently, the ‘shadow object’ gets updated to the current object prior to the most recent transformation.
- Corresponding vertices between the current object and the ‘shadow object’ are connected with translucent edges, to show the exact movement of each vertex.

A Caveat in Chaining Transformations

- Consider maintaining a stack of transformation matrices for transforming a space in $\mathbb{R}^n$. In fact, Visu(AL)-1 does this for most matrices in the $\mathbb{R}^3$ case. Consider the following operations:
 - A variable to hold the current matrix $\mathbf{C}$, a result of pre-multiplying the 3×3 transformation matrices with $\mathbf{I}_3$, starting with the matrices inserted earlier (from the bottom of the stack).
 - A push operation which pushes a new transformation matrix $\mathbf{M}$ onto the stack. Then, update $\mathbf{C}_{after} = \mathbf{M}\mathbf{C}_{before}$.
 - A pop operation which works as follows: It peeks the top of the stack to obtain the most recently pushed transformation matrix $\mathbf{M}$, and then pushes $\mathbf{M}^{-1}$ into the stack using the above-described push operation, so that $\mathbf{C}_{after} = \mathbf{M}^{-1}\mathbf{M}\mathbf{C}_{before} = \mathbf{C}_{before}$. Then, the top two matrices in the stack (which would be $\mathbf{M}^{-1}$ and $\mathbf{M}$) are popped.
- The above stack provides convenience in implementing an ‘undo’ feature for a computer program. However, this assumes every transformation matrix pushed into the stack is invertible, which may not be the case!
- Can you give an example of a non-trivial linear transformation which cannot be reversed? That is, its standard matrix is singular, but it is not the zero matrix. Geometrically, why is this transformation irreversible?

One possible answer: Consider the transformation matrix $\begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}$ which projects any vector in $\mathbb{R}^2$ onto the x-axis. By projecting a vector onto a subspace of a smaller dimension, we lose at least one dimension of information that cannot be recovered, since different vectors in $\mathbb{R}^n$ can be projected to the same vector in a given subspace of $\mathbb{R}^n$ with a smaller dimension. Note that homogenous coordinates cannot save us here either! Try to think of why this is so.

The Resolution

- A second stack is used to hold ‘old current matrices’.
- Transformation matrices which are singular (e.g. projection matrices) are pushed onto the main stack as usual.
- If $\mathbf{M}$ is singular, before updating $\mathbf{C}_{after} = \mathbf{M}\mathbf{C}_{before}$, push $\mathbf{C}_{before}$ onto the second stack.
- When we want to pop a matrix $\mathbf{M}_{top}$ from the main stack, we peek the matrix at the top of the stack and its determinant is calculated.
 - If $\det(\mathbf{M}_{top})$ is not close to 0, then $\mathbf{M}_{top}$ is invertible and the previously described algorithm is applied.
 - If $\det(\mathbf{M}_{top})$ is close to 0, then $\mathbf{M}_{top}$ represents a non-invertible transformation. There should exist a matrix at the top of the second stack, which must be the ‘old current matrix’ right before $\mathbf{M}_{top}$ was pushed onto the main stack. We pop the ‘old current matrix’ from the second stack and restore it as the ‘present current matrix’, then pop $\mathbf{M}_{top}$ from the main stack.
- Although the two-stack solution also works for invertible matrices, assuming that invertible transformations are more common than non-invertible ones, this hybrid solution is more efficient memory-wise.

Total Least Squares: Finding the ‘Best Fit Polygon’ in a 3D Space

- Recall from A-level mathematics that we need at least three points to uniquely specify a plane (an affine space of $\mathbb{R}^3$ with dimension 2). However, given four or more points, not all of them may fit on the same plane!
- In real life, it is unlikely that points will be intentionally chosen to not have a close-fitting plane, but rather, rounding errors may cause slight deviations of points from any plane.
- Let’s define the ‘best fit polygon’ as follows: Let S be an **ordered** set of position vectors in $\mathbb{R}^3$, and for each $\mathbf{s} \in S$, let $\mathbf{s}' \in S'$ be a position vector which lies on some plane P , such that the sum $\sum_S \|\mathbf{s} - \mathbf{s}'\|^2$ is minimised. We square the distances so that both positive and negative deviations are considered; this is similar to the motivation behind a ‘least squares regression line’ from A-level mathematics.
- Then, the ‘best fit polygon’ is the polygon whose **ordered** vertex set is S' . The order of vertices in S' should be ‘the same’ as the order of vertices in S .
- Note: Suppose that a set of vertices specifies a simple (non-self-intersecting) polyhedron. Any projection of this polyhedron onto a plane is not guaranteed to be a simple polygon! The polygon may indeed self-intersect.
- However, in real life practical applications, if you *intended* to specify the vertices of a simple polygon in a given order, but were slightly off due to rounding errors, the corresponding best fit polygon is nearly always simple.

Covariance Matrix

- In statistics, the covariance of two random variables X and Y measures the relationship between the variances of X and Y ; covariance is positive if X and Y vary in the same direction, and negative if in opposite directions.
- It is defined $\text{Cov}(X, Y) = E\{[X - E(X)][Y - E(Y)]\}$. Notice that $\text{Cov}(X, X) = \text{Var}(X)$.
- Another important property: $\text{Cov}(X, Y) = 0$ if X and Y are independent.
- Let $\mathbf{x} = (X_1, X_2, \dots, X_N)^T$ be a random column vector (that is, $X_1, X_2, \dots, X_N$ are random variables). The covariance matrix of $\mathbf{x}$ (which we will denote by $\mathbf{K}$) is defined such that the (i, j) -entry of $\mathbf{K}$ is $\text{Cov}(X_i, X_j)$.
- For example, let $\mathbf{x} = (X_1, X_2)^T$, where X_1 and X_2 are independent random variables with $X_1 \sim \text{Binomial}(42, 0.69)$ and $X_2 \sim \text{Binomial}(1337, 0.67)$. The covariance matrix $\mathbf{K}$ is
$$\begin{pmatrix} \text{Var}(X_1) & 0 \\ 0 & \text{Var}(X_2) \end{pmatrix} = \begin{pmatrix} 8.9838 & 0 \\ 0 & 295.6107 \end{pmatrix}.$$

In the context of geometric 3D points

- Suppose $\{\mathbf{p}_1, \mathbf{p}_2, \dots, \mathbf{p}_N\}$ is a set of position vectors in $\mathbb{R}^3$. We define the centroid of these position vectors, $\bar{\mathbf{p}}$, to simply be their arithmetic mean, $\frac{\mathbf{p}_1 + \mathbf{p}_2 + \dots + \mathbf{p}_N}{N}$.
- Looking at the x , y and z -coordinates of each vector respectively, we can define random variables X , Y and Z to be distributed according to the above set. Then, the above is equivalent to calculating $E(X)$, $E(Y)$ and $E(Z)$ when all vectors are equally likely to be selected.
- Define the vectors $\mathbf{s}_1, \mathbf{s}_2, \dots, \mathbf{s}_N$ by $\mathbf{s}_i = \mathbf{p}_i - \bar{\mathbf{p}}$ for $i = 1, 2, \dots, N$. Then, the product $\mathbf{s}_i \mathbf{s}_i^T$ is:

$$\begin{pmatrix} [X - E(X)]^2 & [X - E(X)][Y - E(Y)] & [X - E(X)][Z - E(Z)] \\ [X - E(X)][Y - E(Y)] & [Y - E(Y)]^2 & [Y - E(Y)][Z - E(Z)] \\ [X - E(X)][Z - E(Z)] & [Y - E(Y)][Z - E(Z)] & [Z - E(Z)]^2 \end{pmatrix}.$$

- And since we also want $\mathbf{s}_1, \mathbf{s}_2, \dots, \mathbf{s}_N$ to be chosen with equal probability, we can define

$$\mathbf{K} = \frac{1}{N}(\mathbf{s}_1 \mathbf{s}_1^T + \mathbf{s}_2 \mathbf{s}_2^T + \dots + \mathbf{s}_N \mathbf{s}_N^T) = \begin{pmatrix} \text{Var}(X) & \text{Cov}(X, Y) & \text{Cov}(X, Z) \\ \text{Cov}(X, Y) & \text{Var}(Y) & \text{Cov}(Y, Z) \\ \text{Cov}(X, Z) & \text{Cov}(Y, Z) & \text{Var}(Z) \end{pmatrix}.$$

That is, $\mathbf{K}$ is a covariance matrix.

In the context of geometric 3D points

- Recall that if $X = \{x_1, x_2, \dots, x_N\}$ and $Y = \{y_1, y_2, \dots, y_N\}$ are coordinates of N data points, then $(\bar{x}, \bar{y})$ lies on the least squares regression line.
- Similarly, the centroid of a set of points in $\mathbb{R}^3$ must lie on any best fit plane. The proof is out of scope of this document, however.
- Let P_i be the point in $\mathbb{R}^3$ with position vector $\mathbf{p}_i$, and let $\Pi : \mathbf{r} \cdot \mathbf{n} = \bar{\mathbf{p}} \cdot \mathbf{n}$ be the best fit plane in $\mathbb{R}^3$ with unit normal $\mathbf{n}$. Then, the distance from P_i to Π is simply $\mathbf{s}_i \cdot \mathbf{n} = \mathbf{s}_i^T \mathbf{n}$.
- Also, notice that $(\mathbf{s}_i^T \mathbf{n})^2 = (\mathbf{s}_i^T \mathbf{n})^T \mathbf{s}_i^T \mathbf{n} = \mathbf{n}^T \mathbf{s}_i \mathbf{s}_i^T \mathbf{n}$, and since we seek to minimise the sum of squared distances, this is equivalent to minimising

$$\mathbf{n}^T \left(\sum_{i=1}^N \mathbf{s}_i \mathbf{s}_i^T \right) \mathbf{n} = N \mathbf{n}^T \mathbf{K} \mathbf{n}.$$

- But since N is just a constant, we are essentially just looking for a unit vector $\mathbf{n}$ such that $\mathbf{n}^T \mathbf{K} \mathbf{n}$ is minimal.

In the context of geometric 3D points

- Since $\mathbf{K}$ is symmetric, it is orthogonally diagonalisable. Let $\{\mathbf{v}_1, \mathbf{v}_2, \mathbf{v}_3\}$ be an orthonormal basis for $\mathbb{R}^3$, which are also eigenvectors of $\mathbf{K}$ with respective eigenvalues λ_1, λ_2 and λ_3 , where $\lambda_3 \leq \lambda_2 \leq \lambda_1$.
- In fact, the eigenvalues are guaranteed to be non-negative. Proof on the next page.
- Any unit vector $\mathbf{n}$ can be written as $a_1\mathbf{v}_1 + a_2\mathbf{v}_2 + a_3\mathbf{v}_3$, where $a_1^2 + a_2^2 + a_3^2 = 1$.
- One can then calculate that $\mathbf{n}^T\mathbf{K}\mathbf{n} = \lambda_1a_1^2 + \lambda_2a_2^2 + \lambda_3a_3^2$.
- We can deduce that $\lambda_3 \leq \mathbf{n}^T\mathbf{K}\mathbf{n} \leq \lambda_1$. The proof is left as an exercise.
- Hence, we see that the total least squares distance is attained when $\mathbf{n}$ is an eigenvector corresponding to the smallest eigenvalue of $\mathbf{K}$.
- The best-fit plane is the plane with normal $\mathbf{n}$ and passing through the centroid of the given set of points.
- Finally, the best-fit polygon for the given set of points is the polygon formed by projecting the points onto the best-fit plane and taking the projected points in the same order.

Proof of non-negative eigenvalues

- Definition: A symmetric matrix $\mathbf{M}$ of order N is positive semi-definite if, for every vector $\mathbf{x}$ in $\mathbb{R}^N$, $\mathbf{x}^T \mathbf{M} \mathbf{x} \geq 0$.
- If $\mathbf{A}$ is a square matrix, then $\mathbf{A}^T \mathbf{A}$ is positive semi-definite; this comes from $\mathbf{x}^T (\mathbf{A}^T \mathbf{A}) \mathbf{x} = (\mathbf{A} \mathbf{x})^T (\mathbf{A} \mathbf{x}) = \|\mathbf{A} \mathbf{x}\|^2 \geq 0$.
- The sum of two positive semi-definite matrices of the same order is also positive semi-definite. Check it yourself!
- Hence, $\mathbf{K}$ is positive semi-definite.
- Let $\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_N$ be eigenvectors of a positive semi-definite matrix $\mathbf{M}$ with respective eigenvalues $\lambda_1, \lambda_2, \dots, \lambda_N$, such that $\{\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_N\}$ is an orthonormal basis for $\mathbb{R}^N$. Then, $\mathbf{v}_i^T \mathbf{M} \mathbf{v}_i = \lambda_i \geq 0$ for $i = 1, 2, \dots, N$.
- Since we have proven that any general positive semi-definite matrix must have only non-negative eigenvalues, the same must be true for $\mathbf{K}$.

Introduction to the Linear Algebra of 2D Image Processing



Digital Image Representation

- The simplest form of a digital image is a black and white image. These images are said to have 1 *channel*, representing the brightness of a pixel.
- The channels in a digital image tells us what data is being stored.
- A digital image can also be coloured, of course. You have likely heard of the RGB colour model. RGB images have 3 channels; red, green and blue.
- Digital images store data values for each pixel based on the brightness or intensity of colour.
- Recall that a binary bit is either 0 or 1, and that an '8-bit' system means that there are $2^8 = 256$ distinct values.
- On most digital images transmitted through modern personal computers, each channel has 8 bits, that is, a single pixel stores 24 bits of data.
- However, some advanced computer programs can recognise images with 16-bit channels, and so each pixel stores 48 bits of data.

Digital Image Representation

- A digital channel is a 2D array of numbers, that is, it is a matrix. Hence, a single-channel image such as a black and white image can be represented using a single matrix.
- A 3-channel image such as an RGB image can either be represented using individual matrices for each channel or by using a single 'matrix' whose entries are 3D vectors instead of scalars.
- The latter representation is known as a tensor.
- Transformations modelled using tensors aim to achieve similar objectives as those using matrices, just generalised to higher dimensions, so don't worry too much.

- Here is a simple 1-channel image with dimensions 5×1 :

0	63	127	191	255
---	----	-----	-----	-----

- Here is a simple 3-channel image with dimensions 4×2 :

0	255	0	0
0	0	255	0
0	0	0	255
255	0	255	255
255	255	0	255
0	255	255	255

Digital Image Representation

- Note that in common parlance, we usually state the dimensions of an image as width $\times$ height, but in terms of matrices, the width is the number of columns, and the height is the number of rows.
- In fact, the matrix representations of the images shown earlier have dimensions 1×5 and 2×4 respectively, so care must be taken to avoid confusion with the common dimensional specification of an image.
- However, for the purpose of LA operations in image processing, it is also common to convert image matrices to column vectors. This is done simply by reading the image from left to right and top to bottom.
- For example, the image $\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}$ becomes $(1\ 2\ 3\ 4\ 5\ 6\ 7\ 8\ 9)^T$.
- By writing $\mathbf{x}$ as a column vector, we can define linear operators that modify $\mathbf{x}$ to give the image $\mathbf{x}'$. Suppose also that some image noise can be written as a column vector $\mathbf{n}$.
- Then, we can form a linear system such as $\mathbf{K}\mathbf{x} + \mathbf{n} = \mathbf{x}'$, where $\mathbf{K}$ is a linear operator.
- Recovering the original image is equivalent to solving the least squares problem to minimise $\|\mathbf{n}\|^2 = \|\mathbf{K}\mathbf{x} - \mathbf{x}'\|^2$.
- This is just one example of how writing an image as a column vector may be useful.

Matrix-Like Operations on Images

- Consider the matrices $\mathbf{C}_1 = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix}$ and $\mathbf{C}_2 = \begin{pmatrix} 0 & 0 \\ 1 & 0 \\ 0 & 1 \end{pmatrix}$, and the image $\mathbf{M} = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}$.
- Calculating $\mathbf{C}_1 \mathbf{M} \mathbf{C}_2$ gives us a sub-matrix of $\mathbf{M}$, $\begin{pmatrix} 2 & 3 \\ 5 & 6 \end{pmatrix}$. That is, we have cropped the image $\mathbf{M}$.
- In general, we can express any crop operation similarly to the above, where $\mathbf{C}_1$ and $\mathbf{C}_2$ choose the rows and columns to keep respectively.
- Next, consider this 4×4 image. We divide the image into 2×2 regions as shown, and then take the average values of the pixels in each region and update all pixels in that region to be that average value.

15	31	47	63
79	95	111	127
143	159	175	191
207	223	239	255



55	55	87	87
55	55	87	87
183	183	215	215
183	183	215	215

- The result is a less detailed image. This process is essentially a simplified version of pixelation.

Questions

- What matrix can be pre-multiplied to each 2×2 region to achieve the above result?
- Is matrix multiplication for pixelation efficient?

Answer: $\frac{1}{4} \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$. Matrix multiplication is inefficient especially for large pixel blocks; we should simply just use arithmetic and array accesses instead since each value is weighted equally.

Activity

- Come up with some small square matrix $\mathbf{M}$ of order n representing an image.
- Define $\mathbf{C}$ to be the matrix such that the (i, j) entry is 1 if $i + j = n + 1$, and 0 otherwise.
- Calculate $\mathbf{CM}$, $\mathbf{MC}$, $\mathbf{CMC}$, $\mathbf{CM}^T$ and $\mathbf{M}^T\mathbf{C}$, and explain the results in terms of image transformations.
- How can these transformations be adapted to non-square images?

Other Ways to Transform Images

- Apart from the earlier transformations shown which can be expressed in terms of matrices, images can also be transformed using colour filters and convolutions.
- In these slides, we will assume that colour filters are applied to a column vector representing a single pixel, and convolutions are applied to simple 2D matrices representing black-and-white images.
- Although convolutions can also be applied to coloured images, here we will focus on black-and-white images, so that it is easier to understand the fundamental concepts first. Once the essence of applied linear algebra has been captured, we can extend it to coloured images.
- Wait... What are filters and convolutions, even!?

Introduction to Colour Filters

- Since an RGB image is made up of pixels, and each pixel can be thought of as a 3D vector, there exist transformations that can be applied to a single pixel, and by extension, an entire RGB image.
- Each entry can be stored as an integer between 0 and 255 (both inclusive) or as a decimal number between 0.0 and 1.0 (both inclusive). We will assume the former for these slides.
- Suppose $\mathbf{p} \in S$ is the vector representing a pixel, and we want to map it to $\mathbf{p}' \in S$. Here, S is a subspace of $\mathbb{R}^3$ and it contains every possible colour.
- As with 3D shapes, a colour filter is a linear transformation $T : \mathbb{R}^3 \rightarrow \mathbb{R}^3$ such that $T(\mathbf{p}) = \mathbf{p}'$. That is, there exists a 3×3 matrix $\mathbf{M}$ such that $\mathbf{M}\mathbf{p} = \mathbf{p}'$. The subspaces which contain all possible vectors $\mathbf{p}$ and $\mathbf{p}'$ are of dimension 3.
- For example, the matrix $\begin{pmatrix} 2 & 0 & 0 \\ 0 & 0.5 & 0 \\ 0 & 0 & 0.5 \end{pmatrix}$ intensifies the redness of an image by increasing the values in the image's red channel. Likewise, it de-intensifies the greenness and the blueness.
- Of course, in practical applications, each coordinate must still lie in the interval $[0, 255]$. Hence, if any coordinate exceeds 255, it will be rounded down to 255, and if any coordinate goes below 0, it will be rounded up to 0.

Examples of Colour Filters

- Image A depicts some roses (mostly red).
- Image B depicts an elephant (mostly gray).
- Images C and D are the results of applying the colour filter whose matrix is

$$\begin{pmatrix} 2 & 0 & 0 \\ 0 & 0.5 & 0 \\ 0 & 0 & 0.5 \end{pmatrix}$$

to Images A and B respectively.

- Images E and F are the results of applying the inverse colour filter, with matrix

$$\begin{pmatrix} 0.5 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 2 \end{pmatrix}$$

to Images C and D respectively.

- What do you notice about the quality of recovery of the images' original colours?

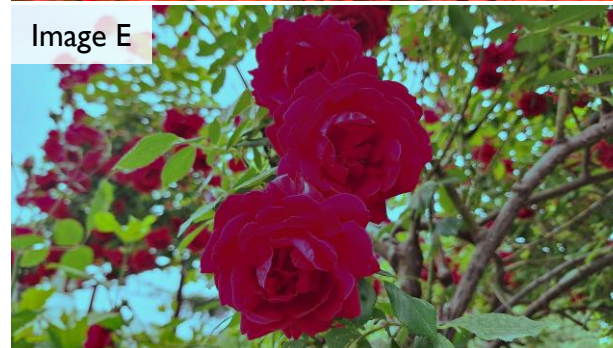
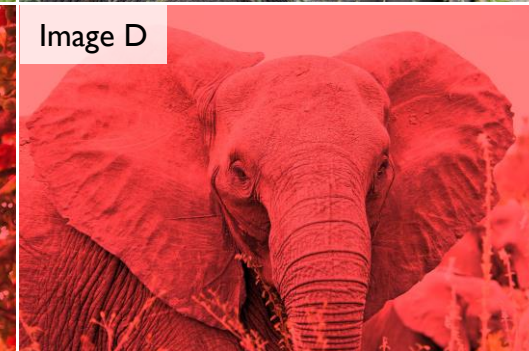


Image A (of roses) is in the public domain.

Image B (of an elephant) is credited to Chris 'Mister-E' Eason, who has licensed his image under CC by 2.0.

Question

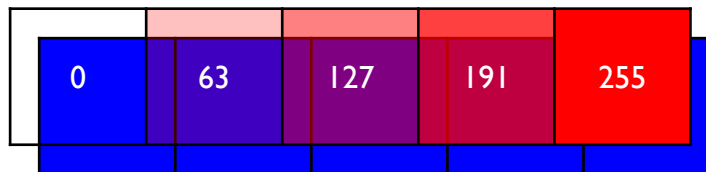
- Do you agree that colour filters with invertible matrix representations are invertible in general?

Affine Colour Filters

- Notice that linear colour filters act on the channels relatively. What if we wanted to act on them absolutely?
- This is conceptually the same as translating along a direction vector in a 3D graphics pipeline. We'll need to introduce homogenous coordinates.
- If $\mathbf{q} = \begin{pmatrix} \mathbf{p} \\ 1 \end{pmatrix}$ then we can solve $\mathbf{M}\mathbf{q} + \mathbf{t} = \mathbf{q}'$ for an affine transformation. Indeed, under the usual homogenous coordinates, $\mathbf{M}$ would be a 4×4 matrix and $\mathbf{q}'$ a 4D vector.
- However, unlike 3D graphics pipelines, it is more common to use a 3×4 matrix $\mathbf{M}$ and let $\mathbf{q}'$ be a 3D vector. This is because, unlike 3D graphics pipelines, the 'fourth dimension' has no real meaning or purpose in image filtering, and just takes up extra space.
- However, to compose a large number of colour filters, indeed, it may be best to keep them as 4×4 matrices, pre-compute the required product, then drop the last row before applying it to the pixel vector.

Alpha Channel

- Although not a feature of physical images, digital image formats in recent years have supported the idea of transparent pixels.
- To do so, on top of the standard RGB colour model, a fourth channel is added. This channel is usually called the ‘alpha’ channel, and thus the colour model which supports transparency is called RGBA.
- An alpha value of 0 indicates complete transparency, and an alpha value of 255 indicates complete opacity. Values in between are translucent to a certain degree accordingly.
- Hence, affine transformations that work on RGBA images would be based on 4×5 matrices.
- Here is a simple example of a foreground image with an alpha channel overlaid over a background image with no alpha channel (i.e. it is fully opaque):



Examples of Colour Filters

- We will assume 3×4 representations of colour filters without an alpha channel.

- Identity Filter: $\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix}$

- Grayscale Filter: $\begin{pmatrix} 0.2126 & 0.7152 & 0.0722 & 0 \\ 0.2126 & 0.7152 & 0.0722 & 0 \\ 0.2126 & 0.7152 & 0.0722 & 0 \end{pmatrix}$ (modern) or $\begin{pmatrix} 0.299 & 0.587 & 0.114 & 0 \\ 0.299 & 0.587 & 0.114 & 0 \\ 0.299 & 0.587 & 0.114 & 0 \end{pmatrix}$ (legacy)

- Sepia Filter: $\begin{pmatrix} 0.131 & 0.534 & 0.272 & 0 \\ 0.168 & 0.686 & 0.349 & 0 \\ 0.189 & 0.769 & 0.393 & 0 \end{pmatrix}$

- Add a Little Yellow to Every Pixel: $\begin{pmatrix} 1 & 0 & 0 & 10 \\ 0 & 1 & 0 & 10 \\ 0 & 0 & 1 & 0 \end{pmatrix}$

- Emphasise Existing Yellow Shades: $\begin{pmatrix} 1.1 & 0 & 0 & 0 \\ 0 & 1.1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix}$

Filter Intensity

- Suppose $\mathbf{I}$ is the identity filter and $\mathbf{F}$ is some other filter we wish to apply to an image, but we want to apply it only at a certain level of intensity.
- For example, we can only apply 30% grayscale filter to slightly desaturate an image while still preserving some colour details.
- The filter matrix is then

$$0.7 \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix} + 0.3 \begin{pmatrix} 0.2126 & 0.7152 & 0.0722 & 0 \\ 0.2126 & 0.7152 & 0.0722 & 0 \\ 0.2126 & 0.7152 & 0.0722 & 0 \end{pmatrix}.$$

- In general, let α be the intensity level we wish to use for a filter, where $0 \leq \alpha \leq 1$. Then the filter matrix is

$$(1 - \alpha)\mathbf{I} + \alpha\mathbf{F}.$$

Question

Colour Inversion filter takes the value of each channel, v_C , and computes $255 - v_C$.

- Write down the matrix $\mathbf{F}$.
- What is the effect of $\alpha = 0.5$?

Answer: $\mathbf{F} = \begin{pmatrix} -1 & 0 & 0 & 255 \\ 0 & -1 & 0 & 255 \\ 0 & 0 & -1 & 255 \end{pmatrix}$. When $\alpha = 0.5$, the filter discards all colour data from the original image and simply returns a solid gray image.

Image convolution kernel

- In image processing, a convolution combines colour information from pixels to form a new image.
- In the new image, each pixel's value is a function of the values of both the corresponding pixel in the original image and nearby pixels in the original image.
- This function is determined by a small matrix, known as the **convolution kernel**. Note that this term has no relation to the kernel of a linear transformation.
- For each pixel in the original image, a convolution kernel is centred at this pixel and the value of the convolution is calculated. The resultant value is the value of the pixel at the corresponding position in the new image.
- Since the convolution kernel uses information from an equal number of rows above and below the centre pixel, and an equal number of columns to the left and right of the centre pixel, it must have **odd** dimensions.
- Hence a convolution kernel can be 1×3 , 5×3 , 7×7 , etc.
- Generally, a convolution kernel is **much smaller** than the image it will be convolved with.
- A convolution kernel is not necessarily square, but we will use square kernels for our first few examples as they are far more common than non-square kernels.

Definition of image convolution

- Suppose now that $\mathbf{P}$ is an $n \times m$ image and $\mathbf{C}$ is a convolution kernel with dimensions $(2h + 1) \times (2k + 1)$.

- Given an image $\mathbf{P} = \begin{pmatrix} p_{1,1} & \cdots & p_{1,m} \\ \vdots & \ddots & \vdots \\ p_{n,1} & \cdots & p_{n,m} \end{pmatrix}$ and a convolution kernel $\mathbf{C} = \begin{pmatrix} c_{1,1} & \cdots & c_{1,2k+1} \\ \vdots & \ddots & \vdots \\ c_{2h+1,1} & \cdots & c_{2h+1,2k+1} \end{pmatrix}$, the value of the pixel (a, b) of the convolved image (provided $h + 1 \leq a \leq n - h$ and $k + 1 \leq b \leq m - k$) is given by

$$\sum_{i=1}^{2h+1} \sum_{j=1}^{2k+1} (p_{a+i-h-1, b+j-k-1}) (c_{i,j})$$

- If the inequalities are not satisfied, this means that the kernel is being applied near the edge of the image, and there are a number of possible ways to resolve this.
- In image processing, the usual approach is to extrapolate the image boundaries for missing pixels using the values of the nearest neighbour that does exist. For example, in the image $\mathbf{P}$, both the pixel directly above and directly to the left of the pixel at $(1, 1)$ will have the value $p_{1,1}$.
- Note that, unless otherwise stated, we allow entries in intermediary output matrices which are not output images to lie outside the interval $[0, 255]$. Only when we process the final image do we need to clip the values accordingly.

Example of image convolution

- For example, consider the following convolution kernel $\mathbf{C} = \frac{1}{9} \begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix}$.
- Consider also the following 5×5 image $\mathbf{P}$ on the left (note that the numbers in red denote extrapolated pixels):

18	18	36	54	36	18	18
18	18	36	54	36	18	18
36	36	54	72	54	36	36
54	54	72	90	72	54	54
36	36	54	72	54	36	36
18	18	36	54	36	18	18
18	18	36	54	36	18	18

convolution

	30	42	48	42	30	
	42	54	60	54	42	
	48	60	66	60	48	
	42	54	60	54	42	
	30	42	48	42	30	

- The convolution of $\mathbf{P}$ with $\mathbf{C}$ at $(2, 2)$ (highlighted in blue) is $\frac{1}{9}(18 + 36 + 54 + 36 + 54 + 72 + 54 + 72 + 90) = 54$.
- The result of convolving the entire image $\mathbf{P}$ is shown on the right. Since all entries lie within the interval $[0, 255]$, the result can immediately be interpreted as another image $\mathbf{Q}$.

Example of image convolution

- For example, consider the following convolution kernel $\mathbf{C} = \begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix}$.
- Consider also the following 5×5 image $\mathbf{P}$ on the left (note that the numbers in green denote extrapolated pixels):

20	20	0	20	20	20	20
20	20	0	20	20	20	20
0	0	0	0	0	0	0
20	20	0	20	20	20	20
20	20	0	20	20	20	20
20	20	0	20	20	20	20
20	20	0	20	20	20	20

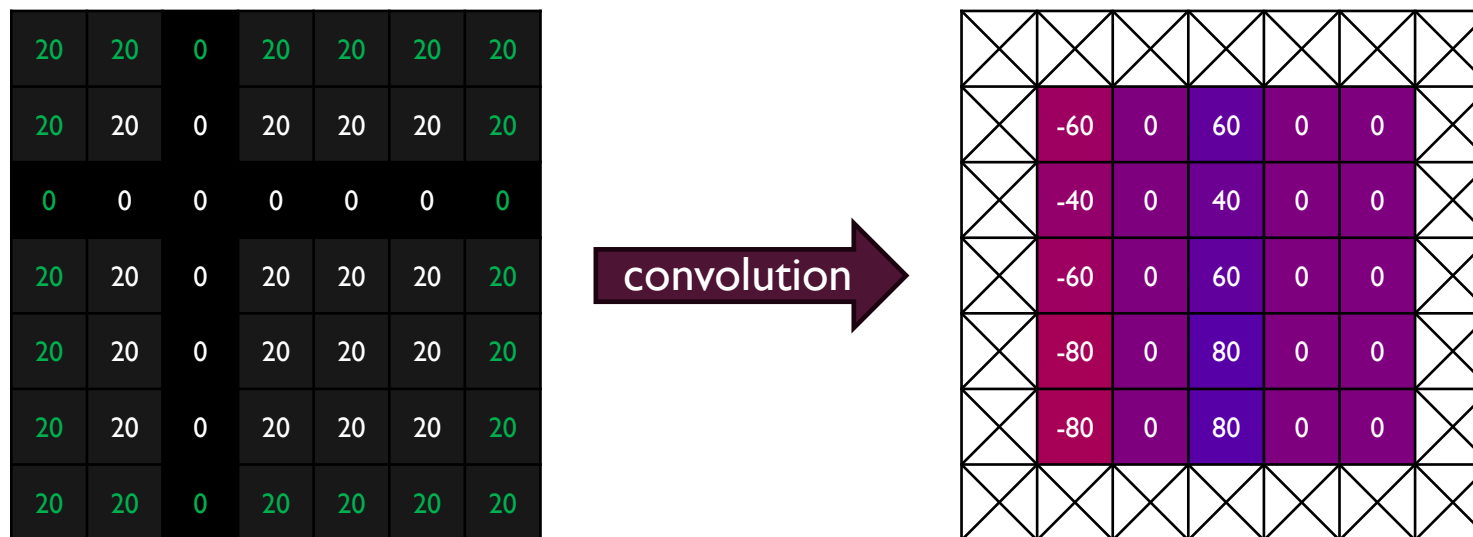
convolution

	-60	0	60	0	0	
	-40	0	40	0	0	
	-60	0	60	0	0	
	-80	0	80	0	0	
	-80	0	80	0	0	

- This time, instead of direct luminosity, we use different shades of purple; those which are closer to red denote negative values and those which are closer to blue denote positive values.
- The result of convolving the entire image $\mathbf{P}$ is shown on the right. As some entries are negative, the result cannot immediately be interpreted as another image $\mathbf{Q}$. If we wanted to do so, all negative values would be changed to 0.

Example of image convolution

- Visually, the convolution below has detected that an edge lies in the second column of our image (not counting the outer ring of extrapolated pixels). The kernel amplifies pixel values of the two outer columns in the 3 x 3 grid, while setting those in the middle column to be 0.



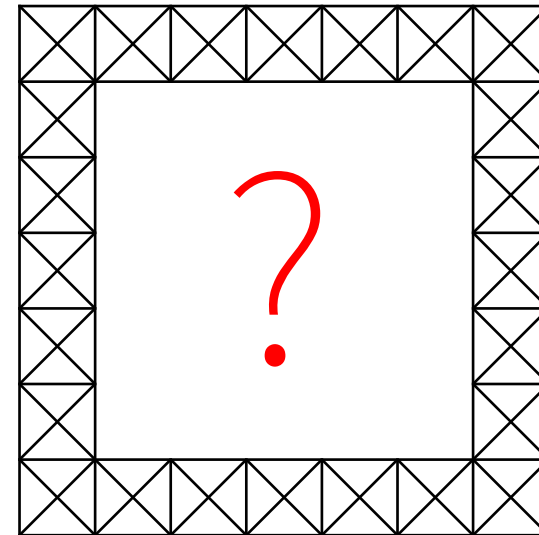
- Indeed, although we used the kernel $\begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix}$, the result would be similar if we used $\begin{pmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{pmatrix}$ instead.
- A computer can infer that, between these two non-zero columns, there is likely a transition between different luminosities in the image.

Exercise

- You are now given the convolution kernel $\mathbf{C} = \begin{pmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{pmatrix}$.
- The image $\mathbf{P}$ is the same as before.

20	20	0	20	20	20	20
20	20	0	20	20	20	20
0	0	0	0	0	0	0
20	20	0	20	20	20	20
20	20	0	20	20	20	20
20	20	0	20	20	20	20
20	20	0	20	20	20	20

convolution



- Find the result of this convolution.

Edge Detection

- Edge detection algorithms aim to isolate sharp changes in colour in an image from more gradual changes.
- There are multiple ways to detect edges. One simple yet well-known operator is the Sobel operator.
- The last two convolutions we just calculated are based on the Sobel kernels, named after Irwin Sobel. The kernels calculate the horizontal and vertical edges of an image.
- The convolution kernels are, as shown before: $\begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix}$, which identifies vertical edges, and $\begin{pmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{pmatrix}$, which identifies horizontal edges.
- Firstly, the image is converted to grayscale, then convolved with both kernels to form two images $\mathbf{I}_x$ and $\mathbf{I}_y$.
- The strength of the edges is defined in this way: The luminosity of each pixel in the final image is the square root of the sum of the squares of the luminosities of the corresponding pixels in $\mathbf{I}_x$ and $\mathbf{I}_y$. They are then either
 - 1) rescaled within the interval $[0, 255]$ (what Visu(AL)-1 does),
 - 2) clipped to the interval $[0, 255]$,
 - 3) clipped to the interval $[0, 255]$ and then a threshold T is imposed so that each pixel with luminosity L is mapped to 255 if $L \geq T$, and 0 otherwise.

Separable Convolutions

- So far, we have seen that colour filters are affine transformations, and we have explored convolutions a little bit. Although the linear algebra concepts in colour filters are analogous to those of 3D graphics transformations, where is the linear algebra in convolutions? You will see it here.
- **Theorem:** A (linear) convolution is separable if its kernel has rank 1 or less.
- The columns of the kernel are all scalar multiples of one another. Let $\mathbf{u} \neq 0$ be any one of these columns. Note that if we cannot find such a $\mathbf{u}$, it implies the kernel has rank 0, which is a trivial case; we now assume that we can indeed find such a $\mathbf{u}$.
- Then, $\mathbf{v}^T$ is a row vector such that each entry going from left to right is the scalar multiple of $\mathbf{u}$ forming the columns of the kernel.
- In general, if we can find non-zero column vectors $\mathbf{u}$ and $\mathbf{v}$ and a non-zero constant k such that $\mathbf{C} = k\mathbf{u}\mathbf{v}^T$, then we must have $\text{rank}(\mathbf{C}) = 1$. This comes from the fact that $\text{rank}(\mathbf{AB}) \leq \min(\text{rank}(\mathbf{A}), \text{rank}(\mathbf{B}))$, if $\mathbf{AB}$ is well-defined.
- Question: Taking $k = 1$, what are $\mathbf{u}$ and $\mathbf{v}$ for the kernel $\begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix}$?

Answer: $\mathbf{u} = \begin{pmatrix} 1 \\ 2 \\ 1 \end{pmatrix}$, $\mathbf{v} = \begin{pmatrix} -1 \\ 0 \\ 1 \end{pmatrix}$ (or both vectors multiplied by -1)

The power of separable convolutions

- The Gaussian kernel is a convolution kernel used for Gaussian blurring. Stay tuned to find out what that is.
- Consider this (approximate) 7×7 Gaussian kernel (sigma = 1.337) **K**:

$$\begin{pmatrix} 0.00058804 & 0.00238117 & 0.00551085 & 0.00728946 & 0.00551085 & 0.00238117 & 0.00058804 \\ 0.00238117 & 0.00964210 & 0.02231517 & 0.02951730 & 0.02231517 & 0.00964210 & 0.00238117 \\ 0.00551085 & 0.02231517 & 0.05164510 & 0.06831333 & 0.05164510 & 0.02231517 & 0.00551085 \\ 0.00728946 & 0.02951730 & 0.06831333 & 0.09036114 & 0.06831333 & 0.02951730 & 0.00728946 \\ 0.00551085 & 0.02231517 & 0.05164510 & 0.06831333 & 0.05164510 & 0.02231517 & 0.00551085 \\ 0.00238117 & 0.00964210 & 0.02231517 & 0.02951730 & 0.02231517 & 0.00964210 & 0.00238117 \\ 0.00058804 & 0.00238117 & 0.00551085 & 0.00728946 & 0.00551085 & 0.00238117 & 0.00058804 \end{pmatrix}$$

- Based on the formula for each value in the Gaussian kernel (again, stay tuned), every column is a scalar multiple of one another, that is, the rank of the kernel is 1.
- This Gaussian kernel can also be written (approximately) as the matrix product $\mathbf{u}\mathbf{u}^T$, where

$$\mathbf{u}^T = (0.02424958 \ 0.09819417 \ 0.22725559 \ 0.30060130 \ 0.22725559 \ 0.09819417 \ 0.02424958).$$

- Convoluting an image with $\mathbf{u}^T$, followed by $\mathbf{u}$, requires approximately 2/7 as many operations as convoluting an image with **K** directly. The efficiency gain is over 70%.

Why Separable Convolutions?

- In the old days of image processing, arithmetic operations on large images were expensive.
- As shown earlier, we improved the efficiency of a Gaussian blur (7×7) kernel by over 70%.
- Hence, if a convolution is separable, then this method is highly useful! The Sobel operator and Gaussian blur are famous examples of separable convolutions.
- How do we separate a separable convolution?
- One obvious method to separating a convolution kernel $\mathbf{K}$ (after proving that $\text{rank}(\mathbf{K}) = 1$) is to take any non-zero column vector as $\mathbf{u}$, compute its magnitude, and use it to find the scalar multiples in $\mathbf{v}$, then compute $\mathbf{K} = \mathbf{u}\mathbf{v}^T$.
- However, we can actually prove that a kernel has rank 1 and determine $\mathbf{u}$ and $\mathbf{v}$ at once! Yet again, we can thank Singular Value Decomposition (SVD) for this.
- Even if a convolution isn't separable, it can often be approximated by separable convolutions. This is a real-world application of the SVD that applies to many fields.

Finding Separable Convolutions

- Recall that SVD reveals the rank of a matrix; the rank of a matrix is exactly equal to the number of non-zero singular values.
- Let $\mathbf{K}$ be the convolution kernel. If we see that $\mathbf{K} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$ has only one non-zero singular value σ , then we can write $\mathbf{K} = \sigma\mathbf{u}\mathbf{v}^T$, where $\mathbf{u}$ is the first column of $\mathbf{U}$ and $\mathbf{v}^T$ is the first row of $\mathbf{V}^T$ (or alternatively, $\mathbf{v}$ is the first column of $\mathbf{V}$, then compute $\mathbf{v}^T$ from there).
- To prove that this does indeed give $\mathbf{K}$ back, one can observe that since $\mathbf{\Sigma}$ has mostly zero rows and columns, they eliminate most of the information from $\mathbf{U}$ and $\mathbf{V}^T$, and the only contributions to the information are actually the first column of $\mathbf{U}$ and the first row of $\mathbf{V}^T$.

Example

- The SVD of $\mathbf{K} = \begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix}$ is $\mathbf{U}\mathbf{\Sigma}\mathbf{V}^T = \begin{pmatrix} -1/\sqrt{6} & \sqrt{5}/6 & 0 \\ -2/\sqrt{6} & \sqrt{2}/15 & -1/\sqrt{5} \\ -1/\sqrt{6} & 1/\sqrt{30} & -2/\sqrt{5} \end{pmatrix} \begin{pmatrix} 2\sqrt{3} & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix} \begin{pmatrix} 1/\sqrt{2} & 0 & -1/\sqrt{2} \\ 1/\sqrt{2} & 0 & 1/\sqrt{2} \\ 0 & 1 & 0 \end{pmatrix}$.
- The above is not the only decomposition, but we usually take the convention that the non-zero singular value is in the (1, 1)-entry of $\mathbf{\Sigma}$. This decomposition comes from MATLAB.
- Notice that $\mathbf{\Sigma}$ can be written as $\mathbf{\Sigma}_1\mathbf{\Sigma}_2$, so that when we calculate $\mathbf{U}\mathbf{\Sigma}_1$, only the first column of $\mathbf{U}$ remains while the others are killed, and when we calculate $\mathbf{\Sigma}_2\mathbf{V}^T$, only the first row of $\mathbf{V}^T$ remains while the others are killed.
- Hence, the zero columns of $\mathbf{U}$ and the zero rows of $\mathbf{V}^T$ can be removed, and thus so can the zero rows and columns of $\mathbf{\Sigma}$. Hence we have found $\mathbf{K} = 2\sqrt{3} \begin{pmatrix} -1/\sqrt{6} \\ -2/\sqrt{6} \\ -1/\sqrt{6} \end{pmatrix} \begin{pmatrix} 1/\sqrt{2} & 0 & -1/\sqrt{2} \end{pmatrix}$.
- In particular, if we choose the (1, 1) entry in $\mathbf{\Sigma}_1$ and $\mathbf{\Sigma}_2$ to be $-\sqrt{6}$ and $-\sqrt{2}$ respectively, then the decomposition is the same as what we have obtained by inspection earlier.

Low-Rank Approximations

- Most convolutions are not separable. However, they can be approximated by low-rank matrices.
- To do this, given a convolution kernel $\mathbf{K}$, we compute the SVD of $\mathbf{K}$ and arrange the singular values in descending order, and find the corresponding orthogonal eigenbases that form $\mathbf{U}$ and $\mathbf{V}^T$.
- The simplified explanation is: Each eigenvector of $\mathbf{K}^T\mathbf{K}$ represents a direction along which data is carried. The larger the eigenvalue, the more important the data carried in the direction of the corresponding eigenvector.
- The n^{th} low-rank approximation of $\mathbf{K}$, $\mathbf{K}_n$, uses the n largest singular values of $\mathbf{K}$ together with the first n columns of $\mathbf{U}$ and the first n rows of $\mathbf{V}^T$ (under the convention that we arrange the eigenvectors in the order corresponding to the decreasing order of the singular values of $\mathbf{K}$.)
- Of course, if $n = \text{rank}(\mathbf{K})$, then the approximation becomes exact.
- Furthermore, if $\mathbf{K}$ is a symmetric matrix, then its orthogonal diagonalisation is closely related to its SVD; this can be used instead since it is generally faster than SVD.
- Roughly speaking, the accuracy of the approximations depends on the magnitude and closeness of the singular values, as we will see in a few examples.

Example

- Consider the edge detection kernel $\mathbf{K} = \begin{pmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{pmatrix}$. Check that $\text{rank}(\mathbf{K}) = 2$, hence not separable. SVD of $\mathbf{K}$ is:

$$\mathbf{U}\mathbf{\Sigma}\mathbf{V}^T \approx \begin{pmatrix} 0.2142 & 0.6739 & -0.7071 \\ -0.9530 & 0.3029 & 0 \\ 0.2142 & 0.6739 & 0.7071 \end{pmatrix} \begin{pmatrix} 4.4495 & 0 & 0 \\ 0 & 0.4495 & 0 \\ 0 & 0 & 0 \end{pmatrix} \begin{pmatrix} -0.2142 & 0.9530 & -0.2142 \\ 0.6739 & 0.3029 & 0.6739 \\ -0.7071 & 0 & 0.7071 \end{pmatrix}.$$

- The first low-rank approximation only uses the first column of $\mathbf{U}$, the first singular value and the first row of $\mathbf{V}^T$:

$$\mathbf{K}_1 = 4.4495\mathbf{u}_1\mathbf{v}_1^T \approx \begin{pmatrix} -0.2041 & 0.9083 & -0.2041 \\ 0.9083 & -4.0413 & 0.9083 \\ -0.2041 & 0.9083 & -0.2041 \end{pmatrix}.$$

- This is fairly close to our original kernel, but the edges produced wouldn't be as sharp.
- Since $\text{rank}(\mathbf{K}) = 2$, we should expect $\mathbf{K}_2$ to give us the exact kernel back. Indeed,

$$\mathbf{K}_2 = 4.4495\mathbf{u}_1\mathbf{v}_1^T + 0.4495\mathbf{u}_2\mathbf{v}_2^T = \begin{pmatrix} -0.2041 & 0.9083 & -0.2041 \\ 0.9083 & -4.0413 & 0.9083 \\ -0.2041 & 0.9083 & -0.2041 \end{pmatrix} + \begin{pmatrix} 0.2041 & 0.0918 & 0.2041 \\ 0.0918 & 0.0412 & 0.0918 \\ 0.2041 & 0.0918 & 0.2041 \end{pmatrix} \approx \mathbf{K}.$$

Example

- Consider the kernel $\mathbf{K} = \begin{pmatrix} 1 & 1 & 1 \\ -1 & 0 & 1 \\ 1 & 0 & 1 \end{pmatrix}$. Check that $\text{rank}(\mathbf{K}) = 3$, hence not separable. SVD of $\mathbf{K}$ is:

$$\mathbf{U}\mathbf{\Sigma}\mathbf{V}^T \approx \begin{pmatrix} -0.7882 & 0 & -0.6154 \\ 0 & -1 & 0 \\ -0.6154 & 0 & 0.7882 \end{pmatrix} \begin{pmatrix} 2.1358 & 0 & 0 \\ 0 & 1.4142 & 0 \\ 0 & 0 & 0.6622 \end{pmatrix} \begin{pmatrix} -0.6572 & -0.3690 & -0.6572 \\ 0.7071 & 0 & -0.7071 \\ 0.2610 & -0.9294 & 0.2610 \end{pmatrix}.$$

- We calculate

$$\sigma_1 \mathbf{u}_1 \mathbf{v}_1^T \approx \begin{pmatrix} 1.1063 & 0.6213 & 1.1063 \\ 0 & 0 & 0 \\ 0.8638 & 0.4851 & 0.8638 \end{pmatrix}, \sigma_2 \mathbf{u}_2 \mathbf{v}_2^T = \begin{pmatrix} 0 & 0 & 0 \\ -1 & 0 & 1 \\ 0 & 0 & 0 \end{pmatrix}, \sigma_3 \mathbf{u}_3 \mathbf{v}_3^T \approx \begin{pmatrix} -0.1063 & 0.3787 & -0.1063 \\ 0 & 0 & 0 \\ 0.1362 & -0.4851 & 0.1362 \end{pmatrix}.$$

- In this case, it is easy to see that $\mathbf{K}_1$ is a poor approximation of $\mathbf{K}$, and $\mathbf{K}_2$ is slightly better but still not great; we need all three singular values for an accurate reconstruction.